

Cifrado Débil (Weak Cryptography): Fundamentos y Contramedidas en Aplicaciones Web

En el ámbito de la ciberseguridad, el **cifrado débil** se refiere a la implementación inadecuada o insuficiente de medidas criptográficas en una aplicación, lo que compromete la seguridad de la información sensible. Esta sesión se centrará en los fallos de cifrado relacionados con la configuración de las aplicaciones web y las mejores prácticas para mitigarlos.

Introducción a los Fallos Criptográficos

El cifrado débil es un problema crítico. De hecho, en el *ranking* **OWASP Top 10**, los fallos criptográficos o fallos de diseño relacionados con la seguridad de los datos suelen ocupar una posición destacada (actualmente, se alinean bajo la categoría de **Fallos Criptográficos**). Estos fallos no suelen ser errores de programación en sí mismos, sino más bien **fallos de configuración del servidor** o de la arquitectura de la aplicación web.

El objetivo primordial del cifrado robusto es proteger la **confidencialidad** y la **integridad** de la información sensible (de clientes, de la organización, etc.). Esta protección debe aplicarse en dos estados fundamentales:

1. **En almacenamiento (At Rest):** Los datos guardados en bases de datos o sistemas de archivos.
2. **En tránsito (In Transit):** Los datos mientras se transmiten a través de la red, como entre el navegador del usuario y el servidor web.

En esta sesión, nos centraremos en la protección de los datos **en tránsito**, analizando dos configuraciones cruciales para prevenir ataques de interceptación: el uso de **HTTPS** y la correcta implementación del **método POST** en las peticiones.

1. La Importancia de HTTPS

HTTPS son las siglas de **Hypertext Transfer Protocol Secure** (Protocolo Seguro de Transferencia de Hipertexto). Es la versión segura del protocolo HTTP, que utilizamos para conectarnos a la gran mayoría de páginas web. Su objetivo principal es asegurar la **confidencialidad** e **integridad** de los datos transmitidos entre el navegador del cliente y el servidor web.

Funcionamiento y Puertos

HTTPS funciona mediante la combinación del protocolo HTTP con una capa de seguridad basada en **SSL/TLS (Secure Sockets Layer/Transport Layer Security)**. Inicialmente, se utilizaba SSL, pero este protocolo quedó obsoleto por vulnerabilidades, siendo reemplazado por **TLS** (cuyo desarrollo comenzó en 1999). Aunque a menudo se sigue mencionando SSL, el estándar actual de seguridad para HTTPS es TLS.

El protocolo HTTPS utiliza por defecto el **puerto 443** para la transferencia de información segura, a diferencia de HTTP, que usa el puerto 80. Si una conexión HTTPS se realizara a través de un puerto no estándar (por ejemplo, <https://ejemplo.com:8443>), el número de puerto debería especificarse explícitamente en la URL.

Prevención de Ataques

El uso de HTTPS establece una **conexión cifrada segura** entre el cliente (navegador) y el servidor. Esto evita que, aunque las comunicaciones sean interceptadas por un tercero, este pueda acceder al contenido legible de las peticiones.

Los ataques que HTTPS ayuda a mitigar son principalmente a nivel de red, como:

- **Man-in-the-Middle (MITM):** Un atacante se interpone entre el cliente y el servidor, retransmitiendo y posiblemente alterando las comunicaciones. El cifrado de HTTPS hace que la información interceptada sea ilegible sin la clave de descifrado.

- **Eavesdropping (Escucha Clandestina):** Consiste en la captura y lectura de pequeños paquetes de datos transmitidos en la red (a menudo mediante *sniffers*) para buscar información sensible. Si no hay cifrado, cualquier dato sensible (como contraseñas o tokens) viaja en texto claro, facilitando este ataque.

Pasos para Auditar la Implementación de HTTPS

Durante una auditoría de seguridad de una aplicación web, es crucial verificar la correcta configuración de HTTPS. Los pasos de revisión incluyen:

1. **Redirección Forzosa:** Al intentar acceder a la aplicación mediante el protocolo **HTTP** (por ejemplo, escribiendo `http://ejemplo.com`), el servidor debe **redirigir automáticamente** al protocolo seguro **HTTPS** (`https://ejemplo.com`).
2. **Puertos Abiertos:** Comprobar si el **puerto 80 (HTTP)** está abierto en el servidor. En caso afirmativo, se debe forzar el acceso por HTTP (por ejemplo, `http://[IP_DEL_SERVIDOR]:80`) para confirmar que la única acción permitida sea la redirección inmediata a HTTPS.
3. **HSTS (HTTP Strict Transport Security):** Se recomienda revisar la configuración de la **cabecera de seguridad HSTS**. Esta cabecera le indica al navegador del cliente que, durante un período de tiempo determinado, solo debe comunicarse con el sitio web utilizando HTTPS. Esta medida también incluye comprobaciones estrictas del certificado SSL/TLS.
4. **Funcionalidades Sensibles: Todas** las funcionalidades que manejen datos sensibles (inicios de sesión, pagos, cambios de contraseña, visualización de datos personales) deben **asegurar** que se ejecutan exclusivamente bajo el protocolo HTTPS.

2. Uso Adecuado del Método POST

El método de petición HTTP utilizado es un factor crucial para la seguridad de los datos en tránsito, incluso cuando se usa HTTPS.

Diferencias entre GET y POST

El protocolo HTTP define varios **métodos de petición o verbos**, siendo **GET** y **POST** los más comunes.

Método GET

- **Propósito Principal:** Se utiliza solamente para **consultar** o solicitar un recurso (por ejemplo, cargar una página o solicitar un conjunto de datos).
- **Transmisión de Parámetros:** Los parámetros (variables) se adjuntan directamente en la **propia URL** (conocido como *Query String*).
- **Seguridad (Confidencialidad):** Es **menos seguro** para la transmisión de datos sensibles, ya que los datos son visibles en la URL, quedan registrados en el historial del navegador, en los *logs* del servidor y pueden ser capturados fácilmente.

Método POST

- **Propósito Principal:** Se utiliza para **crear** o **actualizar** recursos; su función principal es enviar datos al servidor para que los procese o los almacene (por ejemplo, al enviar un formulario de *login* o de registro).
- **Transmisión de Parámetros:** Los parámetros se envían en el **cuerpo de la petición HTTP**, no en la URL.
- **Seguridad (Confidencialidad):** Es **más seguro** para el envío de datos sensibles, dado que los datos no quedan visibles en la URL, el historial del navegador o en los *logs* de acceso del servidor.

Transmisión de Datos en POST

La principal razón por la que el método POST se considera más seguro para el envío de datos sensibles es cómo maneja los parámetros.

Cuando se utiliza **POST**, la información se transmite en el **cuerpo de la petición HTTP**, no en la URL. Esto tiene las siguientes ventajas de seguridad:

- **No Visibilidad en la URL:** Los datos no son visibles para el usuario en la barra de direcciones.

- **No Almacenamiento en Logs/Historial:** La información sensible (como la contraseña) no queda almacenada en el historial del navegador ni en los *logs* de acceso del servidor web.
- **Protección MITM (en combinación con HTTPS):** Aunque un ataque MITM esté en curso, si la conexión es HTTPS, el cuerpo de la petición (donde van los datos) también estará cifrado.

Ejemplo Práctico: Petición POST de un Login

```

1 POST /login.php HTTP/1.1
2 Host: [REDACTED]
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 88
9 Origin: [REDACTED]
10 Connection: close
11 Referer: http://192.168.111.142/login.php
12 Cookie: PHPSESSID=kfgbj4hekbvsse0em2cl32vli1; security=low
13 Upgrade-Insecure-Requests: 1
14
15 username=admin&password=password&Login=Login&user_token=fc2f9202a98d32925f66f1a562264c5a

```

Considere el siguiente ejemplo, extraído de la captura de una petición de *login*:

Análisis de la Petición

La captura de la herramienta de *proxying* (como Burp Suite) muestra una petición HTTP completa para un proceso de inicio de sesión:

Línea 1 (Método y URL):

POST /login.php HTTP/1.1

1.
 - **Método (POST):** Indica que se están enviando datos al servidor para que los procese. Es el método correcto para un *login*.
 - **Recurso (/login.php):** Es la ruta donde el servidor espera recibir los datos de autenticación.
 - **Protocolo (HTTP/1.1):** Versión del protocolo HTTP utilizada. *Nota: En un entorno real y seguro, esta petición debería ser sobre HTTPS para que todo el contenido estuviera cifrado.*

2. **Líneas de Cabecera (Headers):** Contienen metadatos sobre la petición.

- **Host:** [dirección del servidor]
- **User-Agent:** [información sobre el navegador/OS]
- **Content-Type:** `application/x-www-form-urlencoded`: Indica que el cuerpo de la petición contiene datos de formulario codificados como pares clave-valor (como si fueran en la URL, pero en el cuerpo).
- **Content-Length:** `88`: Indica el tamaño del cuerpo de la petición en *bytes*.
- **Cookie:** `PHPSESSID=...`: Muestra el *token* de sesión (PHPSESSID), un dato muy sensible.
- **Referer:** `http://192.168.111.142/login.php`: Indica la URL de la página que originó la petición.

Cuerpo de la Petición (Línea 15): Aquí es donde viajan los datos sensibles.

`username=admin&password=password&Login=Login&user_token=fc2f9202a98d32925f66f1a562264c5a`

3.

- Los datos se envían como pares `clave=valor` separados por `&`.
- **username=admin**: El nombre de usuario introducido.
- **password=password**: La contraseña introducida.
- **user_token=...**: Un token de seguridad, posiblemente un **CSRF Token**, para prevenir ataques de Cross-Site Request Forgery.

Al utilizar POST, los campos `username` y `password` no aparecen en la URL. Si esta conexión fuera **HTTPS**, todo el texto desde el **POST** hasta el final del cuerpo estaría **cifrado**, protegiendo la información de cualquier **Man-in-the-Middle** o **Eavesdropping**.

Pasos para Auditar el Uso de POST

Durante la auditoría, es fundamental verificar que:

1. **Funcionalidades Sensibles: Todas** las funcionalidades que impliquen el envío de datos sensibles al servidor (logins, registros, formularios de contacto con datos personales, cambios de configuración) se realicen **exclusivamente** mediante peticiones **POST**.
2. **Captura con Proxy:** Para revisar el método de petición y la ubicación de los parámetros, se debe utilizar un *proxy* de interceptación como **Burp Suite** o **OWASP ZAP**. Al capturar las peticiones, se puede confirmar si el método usado es **POST** y si los parámetros se encuentran en el cuerpo y no en la URL.

Conclusiones de la Revisión

Hemos analizado por qué la configuración de la criptografía es un deber de conciencia para los desarrolladores y auditores. Los dos puntos clave para asegurar los datos **en tránsito** son:

1. Asegurar que **HTTPS** esté implementado y forzado para **toda** la aplicación.
2. Asegurar que el **método POST** se utilice para **todas** las peticiones que contengan datos sensibles.

Cualquier funcionalidad sensible que no cumpla con estos dos requisitos representa una posible **vulnerabilidad de cifrado débil**, ya que los datos viajarían de una manera insegura o expuesta, lo cual es inaceptable en el manejo de información confidencial.

Cifrado SSL y TLS

Cifrado Débil: Análisis de los Protocolos SSL y TLS

En esta sesión, abordaremos el concepto de **cifrado débil** centrándonos específicamente en los protocolos **SSL** y **TLS**, que son esenciales para asegurar las conexiones web. Analizaremos en qué consisten estos protocolos que se configuran a nivel de servidor, sus diferencias, la evolución de sus versiones, sus vulnerabilidades y el mecanismo fundamental que emplean para establecer una conexión segura.

1. Fundamentos de SSL y TLS

Los protocolos **SSL (Secure Sockets Layer)** y **TLS (Transport Layer Security)** representan diferentes versiones de la tecnología diseñada para mantener seguras las comunicaciones a través de Internet. Su propósito es **proteger cualquier información confidencial** enviada entre dos sistemas (como un navegador y un servidor web), impidiendo que ciberdelincuentes puedan leer o modificar los datos que se transfieren, incluyendo información sensible o personal.

SSL (Secure Sockets Layer) fue el protocolo inicial, naciendo antes de 1995. Actualmente, se considera **totalmente inseguro y obsoleto**.

TLS (Transport Layer Security) es la versión actualizada y más segura que SSL, cuya primera versión (TLSv1.0) apareció en 1999. Es una versión posterior que añade seguridad específicamente en la capa de transporte del modelo de red. Aunque se sigue hablando de "SSL" de forma generalizada debido a su popularidad inicial, **TLS es el estándar de seguridad que se utiliza en la actualidad**.

2. Evolución y Vulnerabilidades de las Versiones

La seguridad en los protocolos de cifrado es dinámica, lo que explica la constante evolución de las versiones.

- Versions:

- SSLv1
- SSLv2
- SSLv3
- TLSv1
- TLSv1.1
- TLSv1.2
- TLSv1.3

Versiones Obsoletas e Inseguras (Nunca usar/Deshabilitar)

Comenzamos con las versiones más antiguas, totalmente obsoletas y que no se deben utilizar.

- **SSLv1, SSLv2 y SSLv3: Nunca** se debe utilizar ninguna versión de SSL. SSLv2 se lanzó en 1995 y SSLv3 en 1996. Esta última nació muy rápido respecto a la anterior porque se encontraron fallos de seguridad. Sin embargo, ambas se han demostrado inseguras y no se recomienda que ningún servidor web las acepte. SSLv3, aunque en principio dejó de utilizarse en 2015, todavía se puede encontrar en servidores antiguos o no actualizados. Si encontramos una página con cualquiera de estos protocolos habilitados, debe reportarse como una vulnerabilidad.
- **TLSv1.0 y TLSv1.1:** Luego llegó **TLSv1.0** en 1999 (el SSL anterior era de 1996). TLSv1.0 se estuvo utilizando hasta 2020, cuando se recomendó evitarlo por ser inseguro. **TLSv1.1** no llegó hasta 2006 (un periodo de 7 años entre versiones) y también se considera inseguro, perdiendo soporte en 2020. Por lo tanto, estos protocolos deberían estar **deshabilitados**.

SSL/TLS

- SSLv1 – **Never use it**
- SSLv2 – **Never use it**
- SSLv3 – **Never use it**
- TLSv1 – **Disabled**
- TLSv1.1 – **Disabled**
- TLSv1.2 – **OK, but...**
- TLSv1.3 – **OK**

Versiones Seguras (Recomendadas)

Las dos versiones más actuales son las únicas que se pueden considerar seguras y deben ser las utilizadas:

- **TLSv1.2:** Llegó en 2008. Se considera seguro, pero la recomendación lleva un "pero" (*OK, but...*). Esto se debe a que debemos tener cuidado con los **tipos de cifrados** que se utilicen, ya que con el cifrado basado en **CBC** pueden empezar a surgir vulnerabilidades. Además, acepta muchas *suítes de cifrado* (lo que le da mucha compatibilidad), pero esto le hace más vulnerable porque puede aceptar algunas que no son lo suficientemente robustas.
- **TLSv1.3:** Lanzada en 2018, es la más actual y es la recomendada (*OK*). Es mucho más segura, tiene **algoritmos de cifrado mejorados**, mayor resistencia a ataques y su proceso de *handshake* es más simple. Esta versión admite solamente **cinco suítes de cifrado** disponibles, y todas se basan en un principio de cifrado autenticado que se considera seguro. Esta simplificación pretende mejorar la seguridad, la eficacia y la rapidez.

El desarrollo de la tecnología está muy vivo y en constante actualización. Ya se está trabajando en la siguiente versión, aunque no se conocen todavía los detalles, se esperan mejoras a nivel de seguridad, rendimiento y compatibilidad. TLS es más seguro y eficiente que SSL, razón por la cual se impuso sobre esta tecnología.

Navegadores como Chrome han forzado al cambio, dejando de ser compatible con SSLv3 en 2014, y con TLSv1.0 y TLSv1.1 en 2020, además de empezar a mostrar notificaciones cuando un servidor aceptaba estos protocolos para indicar su inseguridad.

3. Cifrado Débil: El Protocolo vs. El Certificado

En auditorías web, un fallo en el cifrado débil afecta a la fortaleza del protocolo, pero **no anula la validez del certificado**.

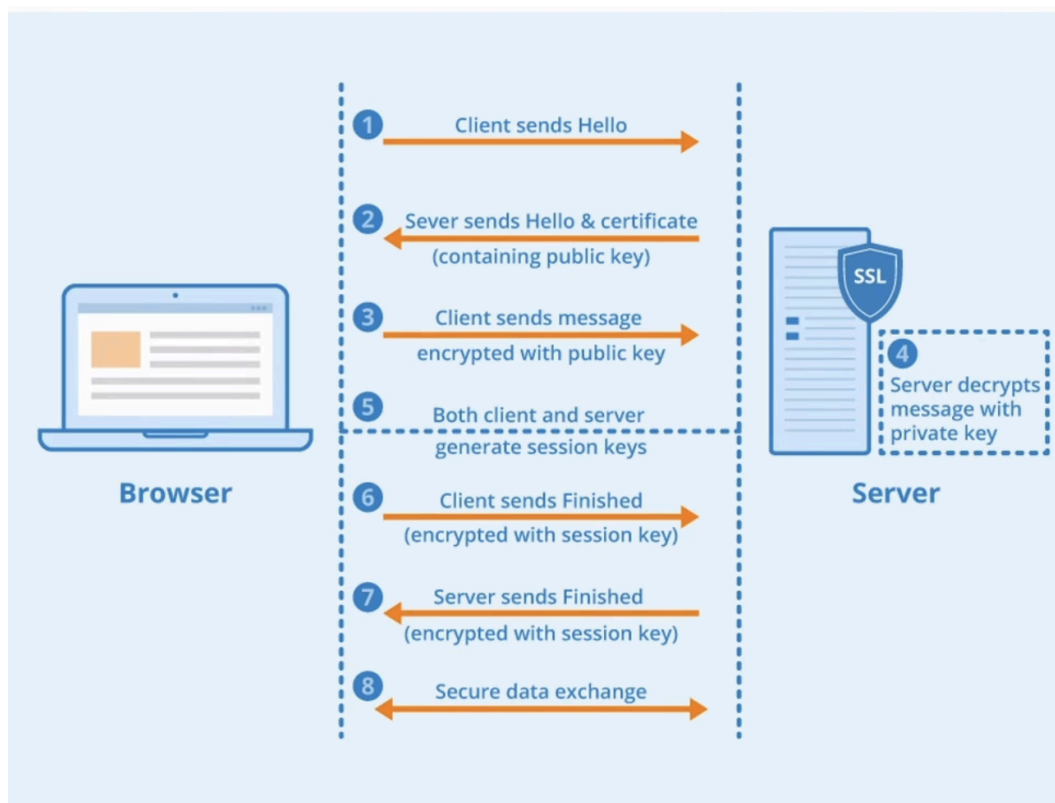
- **Lo que SÍ se ve afectado (El Protocolo / La Conexión):** La debilidad reside en el **protocolo de comunicación (TLS/SSL)** y en las **suítes de cifrado** que negocia el servidor (por ejemplo, usar TLS 1.0 en lugar de TLS 1.3, o usar un

algoritmo obsoleto como DES). Esto es lo que permite a un atacante romper la confidencialidad.

- **Lo que NO se ve afectado (El Certificado):** El **certificado digital** (el archivo `.crt`) sigue siendo técnicamente válido. Un certificado es simplemente un documento que vincula la clave pública a una identidad (el dominio). Su validez depende de la Autoridad Certificadora y su fecha de caducidad, no de si el servidor utiliza un cifrado débil para las conexiones. Es importante tener en cuenta estas diferencias.

4. Funcionamiento Detallado: El TLS Handshake

Una vez que conocemos la parte teórica, veamos cómo funciona TLS.



La imagen de arriba ilustra de forma excelente el proceso de establecimiento de una conexión segura TLS/SSL, conocido como el **"TLS Handshake"** o apretón de manos TLS. Este proceso es fundamental para que se negocie el cifrado antes de que se intercambien datos. Se basa en una serie de mensajes intercambiados entre el Navegador (Cliente) y el Servidor.

El *Handshake* se divide en dos fases: el intercambio de claves asimétricas para establecer la identidad y la clave de sesión, y luego el intercambio de mensajes finales para confirmar el cifrado simétrico.

Fase 1: Intercambio y Generación de Claves

1. **Client Sends Hello (Paso 1):** El Navegador (Cliente) inicia la conexión. Manda un "Client Hello" que incluye la **versión de TLS** y las **suites de cifrado** que admite.
2. **Server Sends Hello & Certificate (Paso 2):** El Servidor responde con su "Server Hello" y el **Certificado SSL/TLS**. En este paso, el navegador y el servidor se ponen de acuerdo en el lenguaje o tipo de cifrado que van a utilizar (que se entienda por ambos). El certificado contiene la **clave pública** del servidor.
3. **Client Sends Message Encrypted with Public Key (Paso 3):** El navegador utiliza la clave pública del certificado para cifrar un mensaje (una clave de sesión pre-maestra) y lo envía de vuelta.
4. **Server Decrypts Message with Private Key (Paso 4):** Solo el servidor posee la **clave privada** correspondiente y la usa para descifrar el mensaje.
5. **Both Client and Server Generate Session Keys (Paso 5):** Después de este intercambio asimétrico, tanto el cliente como el servidor generan la **misma clave de sesión secreta simétrica**. Esta clave se utilizará para cifrar el tráfico masivo de datos por ser mucho más rápida.

Fase 2: Confirmación y Transmisión Segura

1. **Client Sends Finished (Paso 6):** El navegador envía un mensaje "Finished" cifrado con la nueva clave de sesión simétrica. Esto prueba que puede cifrar y descifrar correctamente con la clave compartida.
2. **Server Sends Finished (Paso 7):** El servidor hace lo mismo, enviando su propio mensaje "Finished" cifrado con la clave de sesión, confirmando que el *Handshake* ha sido exitoso.
3. **Secure Data Exchange (Paso 8):** La conexión está completamente establecida, y el intercambio de datos seguros puede comenzar utilizando el cifrado simétrico de la clave de sesión.

Esto es lo que se conoce como *handshake* o apretón de manos, y se produce en la fase inicial (pasos 1 al 7). Lo que hacen es entenderse, buscando el lenguaje de comunicación común para poder enviarse datos que cada uno pueda entender, basándose en que tienen las claves públicas del otro y utilizan el tipo de cifrado que ambos han decidido.

6. Conclusión y Recomendaciones de Seguridad

Como conclusión, hemos visto los protocolos SSL y TLS, y sus versiones. Hemos analizado su evolución, el proceso de *handshake* en TLS y las recomendaciones de seguridad.

Solo se deben considerar seguros y utilizar **TLSv1.2 y TLSv1.3**.

Si un servidor tiene habilitado cualquiera de los demás protocolos (**TLSv1.1**, **TLSv1.0** o cualquier versión de **SSL**), debemos reportarlo como una vulnerabilidad de cifrado débil.

Vulnerabilidades de Cifrado Débil con Nombre Propio (SSL/TLS)

En esta sesión, nos centraremos en el análisis de vulnerabilidades específicas que afectan la seguridad de los protocolos SSL/TLS en los servidores de aplicaciones web. Estas fallas demuestran la importancia de mantener los protocolos actualizados y de configurar adecuadamente las suites de cifrado, ya que comprometen la **confidencialidad** y la **integridad** de los datos.

En general, la explotación exitosa de estas vulnerabilidades busca tres objetivos principales:

1. **Ataque Man-in-the-Middle (MITM):** Interceptar o manipular la comunicación TLS/SSL.
2. **Degradación del Protocolo:** Forzar al servidor o al cliente a utilizar una versión de TLS/SSL más antigua e insegura.
3. **Fuga de Información:** Leer el contenido de la memoria del servidor o descifrar el contenido de las peticiones.

Hemos seleccionado seis de las vulnerabilidades más conocidas y críticas, aunque existen muchas otras (como **DROWN**, **POODLE-TLS**, **BREACH**, **Sweet32** o **Logjam**).

1. BEAST (Browser Exploit Against SSL/TLS)

Fecha de Descubrimiento: 2011 **CVE:** CVE-2011-3389

Concepto y Ataque: **BEAST** explota una debilidad en el modo de operación **Cipher Block Chaining (CBC)** utilizado en **TLS versión 1.0 y todas las versiones anteriores de SSL**.

El ataque permite a un atacante **descifrar gradualmente** los datos cifrados. Lo logra inyectando texto sin un formato conocido en la comunicación y analizando el texto cifrado resultante, permitiéndole inferir el contenido de las peticiones

realizadas al servidor. El ataque afecta la **confidencialidad** de la información transmitida.

Mitigación: Esta vulnerabilidad se considera corregida a partir de **TLSv1.1** y **TLSv1.2**. La corrección se logra mediante el uso de diferentes construcciones de conjuntos de cifrado y, fundamentalmente, utilizando **vectores de inicialización (IVs)** que son aleatorios para cada mensaje, neutralizando la debilidad del modo CBC en estas versiones.

2. Lucky13

Fecha de Descubrimiento: 2013 **CVE:** CVE-2013-0169

Concepto y Ataque: Lucky13 es un **ataque de tiempo criptográfico (timing attack)** dirigido contra las implementaciones de TLS. Explota un problema en el diseño del protocolo que se manifiesta cuando se acepta el modo de **cifrado por bloques (CBC)**, el mismo modo vulnerable en BEAST.

El ataque permite que, al medir las diferencias en el tiempo que le toma al servidor procesar un mensaje cifrado, un atacante pueda obtener información del texto plano subyacente. El nombre **Lucky13** proviene del hecho de que los paquetes TLS cifrados tienen **13 bytes** de encabezado que son consumidos durante uno de los cálculos criptográficos del protocolo.

Mitigación: Aunque es un ataque difícil de explotar en la práctica, su detección y corrección son relativamente sencillas: evitar el uso de *suites de cifrado* que dependan del modo **CBC**.

3. Heartbleed

Fecha de Descubrimiento: 2014 **CVE:** CVE-2014-0160

Concepto y Ataque: Heartbleed es una de las vulnerabilidades más famosas y críticas en la historia de la ciberseguridad, siendo tan importante que incluso se le asignó su propio logo (un corazón del que caen gotas, simbolizando una hemorragia de datos).

Este fallo no se encuentra en el diseño del protocolo TLS/SSL en sí, sino en una biblioteca de código abierto específica: **OpenSSL**, la implementación de TLS más ampliamente utilizada. La vulnerabilidad reside en una implementación defectuosa de la extensión *Heartbeat* (latido).

La explotación de Heartbleed permite a los atacantes **leer información confidencial directamente de la memoria del servidor**. Esto puede incluir datos sumamente sensibles como **claves privadas del servidor**, **claves de sesión**, credenciales de usuario y otros datos personales que se encuentren en la memoria en ese momento. Al explotarse, provoca una **fuga de información** del contenido almacenado en la memoria.

Mitigación: La solución fue el lanzamiento de una nueva versión de **OpenSSL con un parche** que corregía el fallo. Los servidores afectados tenían que realizar dos acciones críticas:

1. **Actualizar** el software OpenSSL inmediatamente.
2. **Revocar y reemplazar** cualquier certificado digital que pudiese haber sido comprometido, dado que las claves privadas podrían haber sido robadas.

4. POODLE (Padding Oracle On Downgraded Legacy Encryption)

Fecha de Descubrimiento: 2014 **CVE:** CVE-2014-3566

Concepto y Ataque: **POODLE** es una vulnerabilidad que afecta específicamente a la versión obsoleta **SSLv3**. Se cataloga como un ataque de tipo **Man-in-the-Middle (MITM)**.

El ataque se basa en un oráculo de *padding* (relleno) que permite a un atacante, al manipular el relleno de los bloques de cifrado, descifrar *byte a byte* la información. El principal riesgo de POODLE es su capacidad para forzar una **degradación** del protocolo, obligando al cliente y al servidor a comunicarse a través del inseguro **SSLv3**, permitiendo así al atacante conocer el contenido de las peticiones y de la información sensible enviada.

5. Ticketbleed

Fecha de Descubrimiento: 2016 **CVE:** CVE-2016-9244

Concepto y Ataque: Ticketbleed es una vulnerabilidad más reciente que también posee su propio icono. Es muy similar a Heartbleed en la forma de actuar, pero afecta a una implementación diferente: la de **F5 BIG-IP** (dispositivos utilizados comúnmente como *load balancers* o *reverse proxies*).

La vulnerabilidad se manifestaba en la función de **conexiones rápidas de TLS**, permitiendo obtener información sensible (fuga de información) de la memoria, similar a la dinámica crítica de Heartbleed.

6. Raccoon

Fecha de Descubrimiento: 2020 **CVE:** CVE-2020-1968

Concepto y Ataque: **Raccoon** es la vulnerabilidad más reciente con nombre propio de esta selección y afecta a la seguridad en el intercambio de claves de **Diffie-Hellman (DH)** durante el proceso de *handshake* en **TLS versión 1.2 y anteriores**.

Esta vulnerabilidad es la razón principal por la que **TLSv1.2** lleva un "pero" en su recomendación de seguridad. Cuando los servidores utilizan el intercambio de claves de Diffie-Hellman, pueden ser vulnerables a Raccoon, lo que permite descifrar la conexión entre cliente y servidor.

La falla se encuentra en la forma en que se maneja una clave crucial llamada *premaster secret*, que se utiliza para generar las claves de cifrado de sesión. La explotación de Raccoon es compleja; requiere realizar una gran cantidad de peticiones y su éxito depende directamente de la configuración específica del servidor. No obstante, al permitir la ruptura de la confidencialidad, hace que **TLSv1.2 no se pueda considerar 100% segura** en todas sus configuraciones.

Mitigación: Debido a esta vulnerabilidad, se recomienda **evitar** la utilización de *suites de cifrado* que empleen los algoritmos de **Diffie-Hellman (DH)** cuando se utilice TLSv1.2.

Conclusiones y Pruebas en Auditorías

Las seis vulnerabilidades analizadas son solo una muestra (como BEAST, Lucky13, Heartbleed, POODLE, Ticketbleed y Raccoon), existiendo otras notables como **BREACH**, **Sweet32** o **Logjam**. Todas estas fallas afectan directamente la seguridad de los protocolos SSL/TLS.

Para proporcionar el mayor grado de seguridad:

- La recomendación actual es **utilizar siempre TLSv1.3**.
- Si se utiliza **TLSv1.2**, es imperativo **revisar las suites de cifrado** que admite y **evitar** cualquier clave de cifrado que utilice los algoritmos de **Diffie-Hellman**.

En una auditoría, estas vulnerabilidades se pueden descubrir analizando la conexión entre el navegador y el servidor de la aplicación, utilizando herramientas que fuerzan la negociación de protocolos y suites de cifrado para identificar configuraciones débiles o vulnerables.

Detección de Vulnerabilidades de Cifrado Débil:

Herramientas de Auditoría Web

En el proceso de auditoría de ciberseguridad, es fundamental contar con herramientas robustas que permitan identificar vulnerabilidades que afecten a los protocolos SSL/TLS, sus versiones, y las *suites de cifrado* utilizadas por un servidor de aplicaciones web.

A continuación, exploraremos tres de las herramientas más efectivas en esta tarea: **Qualys SSL Labs**, **testssl.sh**, y **sslyze**. También mencionaremos otras herramientas útiles para complementar el análisis.

1. SSL Labs (Qualys)

Tipo: Aplicación web (Servicio externo). **Propósito:** Realiza un análisis profundo y no invasivo de la configuración de seguridad SSL/TLS de cualquier servidor web accesible públicamente. La principal ventaja es que **la petición no sale desde la máquina o IP del auditor**, sino desde los servidores de Qualys.

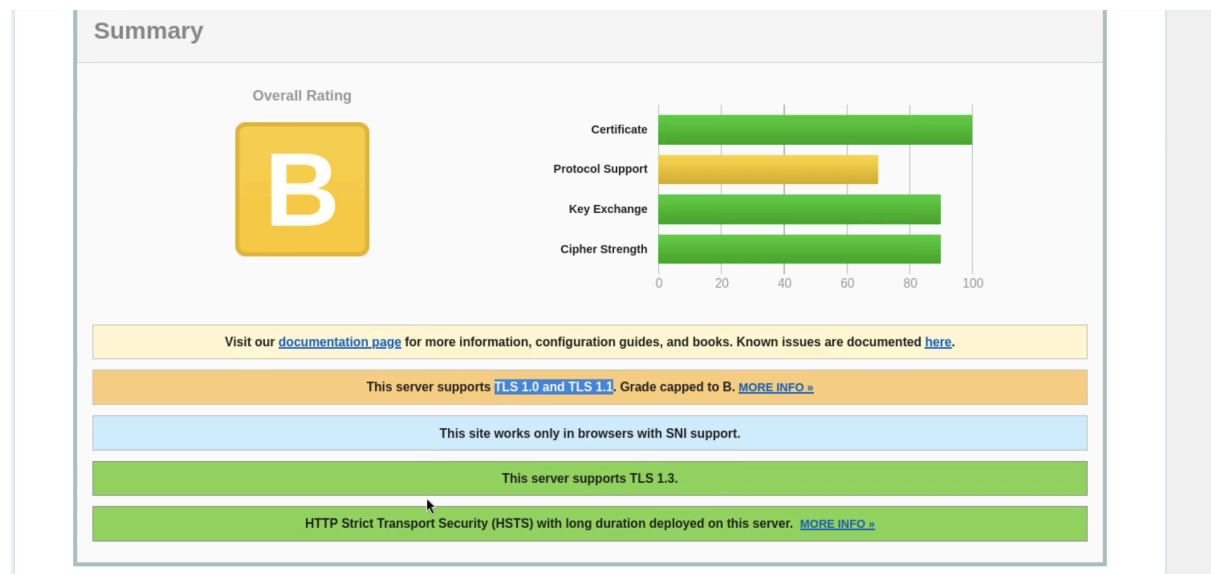
Procedimiento Práctico y Análisis de Resultados

Para utilizar SSL Labs, el auditor debe acceder a la herramienta a través de un navegador web.

1. **Acceso y Configuración:** El auditor navega a la página del **SSL Server Test** de Qualys. Se introduce la URL del objetivo (por ejemplo, 0xword.com). Es crucial marcar la casilla **"Do not show the results on the boards"** para evitar que los resultados del análisis queden expuestos en los paneles públicos que muestran las búsquedas recientes, los mejores y los peores resultados.
2. **Ejecución:** Al hacer clic en *submit*, las peticiones se lanzan desde el servidor de Qualys. La herramienta resuelve la IP pública del dominio y comienza a testear la configuración.

3. **Proceso de Análisis:** Durante la ejecución, SSL Labs testa diferentes aspectos, desde la compatibilidad de protocolos hasta la simulación de *handshakes* con varios *user agents* y la verificación de vulnerabilidades conocidas (como POODLE).

Una vez que el análisis finaliza, la herramienta proporciona un informe con una calificación general, donde **"A" es la mejor nota posible**.



Interpretación del Resultado (Ejemplo): El resultado en la imagen muestra una calificación global de **B**. La evaluación se desglosa en cuatro componentes:

- **Certificate:** La validez y calidad del certificado (100% en el ejemplo).
- **Protocol Support:** La compatibilidad con versiones de protocolo.
- **Key Exchange:** La robustez del intercambio de claves.
- **Cipher Strength:** La solidez de las *suites de cifrado* aceptadas.

En el ejemplo analizado, el servidor acepta **TLSv1.0 y TLSv1.1**, lo cual es la razón principal de que la calificación baje a **B** (ya que estas versiones están deprecadas).

- **Configuración Detallada:** El apartado de *Configuration* confirma que aunque el servidor soporta **TLSv1.3 y TLSv1.2**, también acepta las versiones no recomendadas **TLSv1.1 y TLSv1.0**. Esto debe ser reportado, al menos, como una vulnerabilidad de nivel informativo o bajo, dependiendo de la política del

cliente. También se observa que tiene configurada la cabecera de seguridad **HSTS (HTTP Strict Transport Security)**, lo cual es una buena práctica.



Cipher Suites			
# TLS 1.3 (server has no preference)			
TLS_AES_128_GCM_SHA256 (0x1301)	ECDH x25519 (eq. 3072 bits RSA)	FS	128
TLS_AES_256_GCM_SHA384 (0x1302)	ECDH x25519 (eq. 3072 bits RSA)	FS	256
TLS_CHACHA20_POLY1305_SHA256 (0x1303)	ECDH x25519 (eq. 3072 bits RSA)	FS	256
# TLS 1.2 (suites in server-preferred order)			
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)	ECDH x25519 (eq. 3072 bits RSA)	FS	128
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)	ECDH x25519 (eq. 3072 bits RSA)	FS	256
TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 (0x9e)	DH 2048 bits	FS	128
TLS_DHE_RSA_WITH_AES_256_GCM_SHA384 (0x9f)	DH 2048 bits	FS	256
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (0xc027)	ECDH x25519 (eq. 3072 bits RSA)	FS	WEAK 128
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)	ECDH x25519 (eq. 3072 bits RSA)	FS	WEAK 128
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (0xc028)	ECDH x25519 (eq. 3072 bits RSA)	FS	WEAK 256
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)	ECDH x25519 (eq. 3072 bits RSA)	FS	WEAK 256
TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 (0x67)	DH 2048 bits	FS	WEAK 128
TLS_DHE_RSA_WITH_AES_128_CBC_SHA (0x33)	DH 2048 bits	FS	WEAK 128
TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (0x6b)	DH 2048 bits	FS	WEAK 256
TLS_DHE_RSA_WITH_AES_256_CBC_SHA (0x39)	DH 2048 bits	FS	WEAK 256
TLS_RSA_WITH_AES_128_GCM_SHA256 (0x9c)		WEAK	128
TLS_RSA_WITH_AES_256_GCM_SHA384 (0x9d)		WEAK	256
TLS_RSA_WITH_AES_128_CBC_SHA256 (0x3c)		WEAK	128
TLS_RSA_WITH_AES_256_CBC_SHA256 (0x3d)		WEAK	256

- **Análisis de Cipher Suites:** La revisión de las **suites de cifrado** muestra que TLS 1.3 acepta 3 de las 5 *suites* disponibles. Sin embargo, en TLS 1.2, se listan múltiples *suites*, y varias están marcadas como **WEAK (Débil)**, generalmente por utilizar el modo **CBC** (vulnerable a ataques como BEAST y Lucky13) o por la longitud de sus claves (128 bits, aunque algunas son seguras si usan AES, si van con CBC se vuelven vulnerables). También se notan *suites* que no usan Diffie Hellman, lo cual es relevante por la vulnerabilidad Raccoon.
- **Vulnerabilidades:** La herramienta realiza *checks* automáticos. En el ejemplo, la mayoría de las vulnerabilidades conocidas no afectan directamente al servidor.

2. testssl.sh

Tipo: Script Bash (Herramienta de terminal). **Propósito:** Es una herramienta *open-source* muy popular, utilizada para realizar pruebas de seguridad integrales de

SSL/TLS, protocolos, *suites de cifrado* y vulnerabilidades. Es la favorita de muchos auditores por su **alta visibilidad y el uso intuitivo de colores** en la terminal.

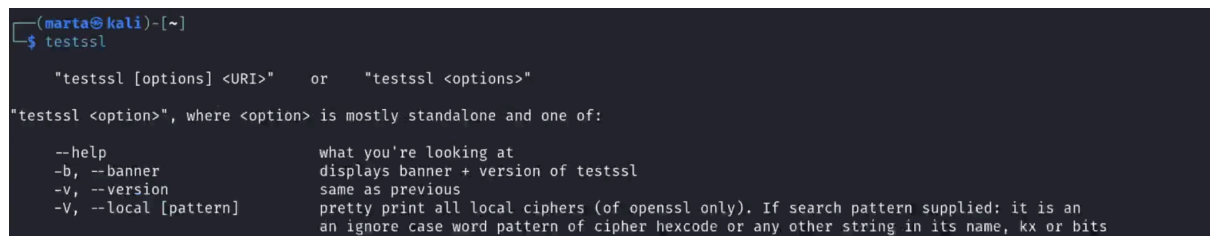
Instalación y Sintaxis del Comando

Dado que **testssl.sh** no suele venir instalada por defecto en Kali Linux, se debe instalar mediante el siguiente comando:

```
sudo apt-get install testssl.sh
```

Una vez instalada, la sintaxis básica del comando es la siguiente:

```
testssl [opciones] <URI>
```



```
(marta@kali)-[~]
$ testssl

"testssl [options] <URI>" or "testssl <options>"

"testssl <option>", where <option> is mostly standalone and one of:

--help          what you're looking at
-b, --banner    displays banner + version of testssl
-v, --version   same as previous
-V, --local [pattern] pretty print all local ciphers (of openssl only). If search pattern supplied: it is an
an ignore case word pattern of cipher hexcode or any other string in its name, kx or bits
```

Sintaxis en detalle: Simplemente se introduce **testssl** seguido de la URL objetivo, por ejemplo:

```
testssl 0xword.com
```

- **Opciones:** El comando acepta diversas opciones (como se ve en la imagen de ayuda). Por ejemplo, el *flag* **-U** revisa exhaustivamente todas las vulnerabilidades conocidas, aunque al ejecutarlo por defecto ya proporciona un análisis muy completo.

Procedimiento Práctico y Análisis de Resultados

Al ejecutar el comando, **testssl.sh** comienza a mostrar el análisis directamente en la consola de comandos de Kali Linux.

1. **Revisión de Protocolos:** Lo primero que revisa son los protocolos. En la salida, las versiones obsoletas, como **TLS 1.0 y TLS 1.1**, aparecen resaltadas en **naranja** (como se ve en la imagen de *testing protocols*). Los protocolos no ofrecidos (como SSLv2 y SSLv3) aparecen en verde.

2. **Categorías de Cifrado:** Revisa las categorías de cifrado, indicando si se aceptan cifrados inseguros. Por ejemplo, si acepta **Obsoleted CBC ciphers**, ya advierte de posibles vulnerabilidades. También verifica que categorías como **NULL ciphers (sin cifrado)**, **Anonymous NULL Ciphers (sin autenticación)**, **Export ciphers** o **LOW: 64 Bit** no sean ofrecidas.
3. **Vulnerabilidades:** La herramienta pasa a revisar automáticamente una gran cantidad de vulnerabilidades con nombre propio (Heartbleed, CCS, Ticketbleed, ROBOT, POODLE, SWEET32, DROWN, LOGJAM, BEAST, LUCKY13, etc.).

```

Testing vulnerabilities
Heartbleed (CVE-2014-0160)      not vulnerable (OK), no heartbeat extension
CCS (CVE-2014-0224)            not vulnerable (OK)
Ticketbleed (CVE-2016-9244), experiment. not vulnerable (OK)
ROBOT                          not vulnerable (OK)
Secure Renegotiation (RFC 5746) supported (OK)
Secure Client-Initiated Renegotiation not vulnerable (OK)
CRIME, TLS (CVE-2012-4929)     not vulnerable (OK)
BREACH (CVE-2013-3587)        potentially NOT ok, "gzip" HTTP compression detected. - only supplied "/" tested
                                Can be ignored for static pages or if no secrets in the page
POODLE, SSL (CVE-2014-3566)    not vulnerable (OK), no SSLv3 support
TLS_FALLBACK_SCSV (RFC 7507)  Downgrade attack prevention supported (OK)
SWEET32 (CVE-2016-2183, CVE-2016-6329) not vulnerable (OK)
FREAK (CVE-2015-0204)         not vulnerable (OK)
DROWN (CVE-2016-0800, CVE-2016-0703) not vulnerable on this host and port (OK)
                                make sure you don't use this certificate elsewhere with SSLv2 enabled services, see
                                https://search.censys.io/search?resource=hosts&virtual_hosts=INCLUDE&q=73B2CD97065A071BE6574AADE34F936
35C88B52C132E4ECDD6C954A6AAE9460C
LOGJAM (CVE-2015-4000), experimental not vulnerable (OK): no DH EXPORT ciphers, no common prime detected
BEAST (CVE-2011-3389)         TLS1: ECDHE-RSA-AES128-SHA ECDHE-RSA-AES256-SHA DHE-RSA-AES128-SHA DHE-RSA-AES256-SHA AES128-SHA
                                AES256-SHA DHE-RSA-CAMELLIA256-SHA DHE-RSA-CAMELLIA128-SHA CAMELLIA256-SHA CAMELLIA128-SHA
                                VULNERABLE -- but also supports higher protocols TLSv1.1 TLSv1.2 (likely mitigated)
                                potentially VULNERABLE, uses cipher block chaining (CBC) ciphers with TLS. Check patches
LUCKY13 (CVE-2013-0169), experimental not vulnerable (OK)
Winshock (CVE-2014-6321), experimental no RC4 ciphers detected (OK)
RC4 (CVE-2013-2566, CVE-2015-2808)

```

- **Interpretación del Resultado de Vulnerabilidades:**
 - **Verde (OK):** El servidor no es vulnerable.
 - **Naranja (Potentially VULNERABLE):** Podría ser vulnerable, o la vulnerabilidad existe en protocolos obsoletos que soporta. Por ejemplo, en el test de **BEAST**, la herramienta indica que la vulnerabilidad existe porque el servidor admite el protocolo **TLS 1.0**. Sin embargo, agrega que también soporta protocolos más altos (**TLSv1.1 y TLSv1.2**), lo que significa que el ataque solo sería posible si se logra un *downgrade* (una bajada de versión) de TLS.
 - **Rojo (VULNERABLE):** La vulnerabilidad existe y es explotable en la configuración actual del servidor.

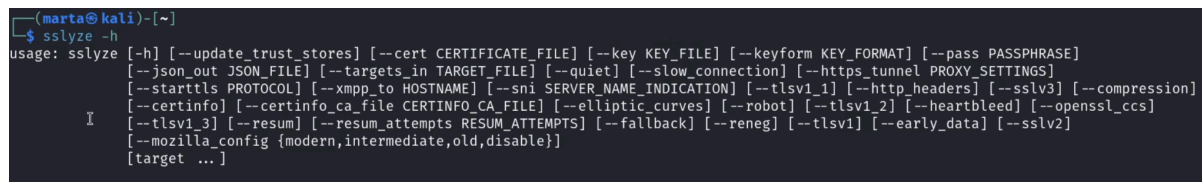
3. sslyze

Tipo: Herramienta Python (Terminal). **Propósito:** Se especializa en la exposición detallada de las *suites de cifrado* habilitadas para cada protocolo individual. Es excelente para obtener una visión rápida y precisa de la compatibilidad de *suites*.

Instalación y Sintaxis del Comando

La herramienta **sslyze** suele venir preinstalada en Kali Linux. Para ver su ayuda y opciones, se usa:

sslyze -h

A terminal window showing the help output for the sslyze command. The prompt is (marta@kali)-[~]. The command entered is sslyze -h. The output lists various options for the tool, including flags for updating trust stores, specifying certificate and key files, setting key formats and passwords, outputting JSON, specifying targets, quiet mode, slow connection, HTTPS tunnel proxy settings, starttls protocol, xmpp to hostname, sni server name indication, TLS versions, HTTP headers, SSL versions, compression, certificate info, certificate info CA file, elliptic curves, robot mode, TLS versions, heartbleed, openssl CCS, TLS versions, resum, resum attempts, fallback, renegotiation, TLS versions, early data, SSL versions, mozilla config, and target.

```
(marta@kali)-[~]
$ sslyze -h
usage: sslyze [-h] [--update_trust_stores] [--cert CERTIFICATE_FILE] [--key KEY_FILE] [--keyform KEY_FORMAT] [--pass PASSPHRASE]
              [--json_out JSON_FILE] [--targets_in TARGET_FILE] [--quiet] [--slow_connection] [--https_tunnel PROXY_SETTINGS]
              [--starttls PROTOCOL] [--xmpp_to HOSTNAME] [--sni SERVER_NAME_INDICATION] [--tlsv1_1] [--http_headers] [--ssl3] [--compression]
              [--certinfo] [--certinfo_ca_file CERTINFO_CA_FILE] [--elliptic_curves] [--robot] [--tlsv1_2] [--heartbleed] [--openssl_ccs]
              [--tlsv1_3] [--resum] [--resum_attempts RESUM_ATTEMPTS] [--fallback] [--reneg] [--tlsv1] [--early_data] [--ssl2]
              [--mozilla_config {modern,intermediate,old,disable}]
              [target ...]
```

Sintaxis en detalle: La forma más sencilla de ejecutar la herramienta es indicando directamente el *host* o dominio a analizar:

sslyze 0xword.com

Procedimiento Práctico y Análisis de Resultados

sslyze es muy rápida en devolver la información.

1. **Conectividad y Certificado:** Primero verifica la conectividad (IP pública, puerto 443) y valida el certificado (que coincida con el *hostname* y que sea válido).
2. **Revisión por Protocolo:** Luego, revisa el soporte y las *suites de cifrado* por versión de protocolo, desde SSL 2.0 en adelante.

```

* SSL 2.0 Cipher Suites:
  Attempted to connect using 7 cipher suites; the server rejected all cipher suites.

* SSL 3.0 Cipher Suites:
  Attempted to connect using 80 cipher suites; the server rejected all cipher suites.

* TLS 1.0 Cipher Suites:
  Attempted to connect using 80 cipher suites.

The server accepted the following 10 cipher suites:
TLS_RSA_WITH_CAMELLIA_256_CBC_SHA          256
TLS_RSA_WITH_CAMELLIA_128_CBC_SHA          128
TLS_RSA_WITH_AES_256_CBC_SHA               256
TLS_RSA_WITH_AES_128_CBC_SHA               128
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA         256      ECDH: prime256v1 (256 bits)
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA         128      ECDH: prime256v1 (256 bits)
TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA      256      DH (2048 bits)
TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA      128      DH (2048 bits)
TLS_DHE_RSA_WITH_AES_256_CBC_SHA           256      DH (2048 bits)
TLS_DHE_RSA_WITH_AES_128_CBC_SHA           128      DH (2048 bits)

The group of cipher suites supported by the server has the following properties:
Forward Secrecy      OK - Supported
Legacy RC4 Algorithm  OK - Not Supported

```

- **Ejemplo de TLS 1.0:** La herramienta intenta conectarse utilizando una gran cantidad de *cipher suites* (80 en la imagen de ejemplo). En la salida, se listan explícitamente las **10 suites** que el servidor **aceptó** para ese protocolo. También informa si el servidor soporta **Forward Secrecy** y si admite el obsoleto algoritmo **RC4** (en el ejemplo, *Not Supported*, lo cual es OK).
- **Comparación de Output:** La principal diferencia entre **sslyze** y **testssl.sh** es visual: mientras que **testssl.sh** utiliza colores para una lectura intuitiva y rápida, **sslyze** muestra el resultado de las pruebas de vulnerabilidades en blanco (OK), requiriendo que el auditor lea la información de manera más detallada.

Conclusiones y Herramientas Complementarias

Hemos aprendido a revisar las configuraciones de seguridad relacionadas con los cifrados SSL/TLS utilizando tres herramientas clave. **SSL Labs** proporciona un análisis externo y una calificación intuitiva. **testssl.sh** ofrece un análisis completo y altamente visual desde la terminal. **sslyze** destaca por la rapidez y el detalle en la enumeración de las *cipher suites* por protocolo.

Existen otras herramientas que pueden complementar este análisis, como:

- **SSLScan:** Una herramienta que viene preinstalada en Kali Linux y se especializa en enumerar las *cipher suites* aceptadas.
- **Nmap:** El escáner de red incluye *scripts* específicos para auditoría SSL/TLS, como:
 - **ssl-cert:** Para obtener información detallada del certificado.

- `ssl-enum-ciphers`: Para listar los algoritmos de cifrado aceptados por el servidor.

Ejercicio Práctico: Auditoría de Cifrado Débil y Detección de Vulnerabilidades SSL/TLS

Esta sesión se centra en la aplicación práctica de los conocimientos adquiridos sobre cifrado débil y protocolos SSL/TLS. El ejercicio propuesto exige el uso de múltiples herramientas de auditoría para comparar resultados y evaluar la configuración de seguridad de dos dominios web reales.

Enunciado del Ejercicio

El objetivo del ejercicio es instalar y ejecutar dos herramientas de línea de comandos, junto con una herramienta web, sobre dos dominios de prueba para identificar fallos en su configuración criptográfica.

Exercise

- Install in your VM:
 - Testssl.sh
 - Sslscan
- Execute them over:
 - mypublicinbox.com
 - keysmartinc.com
- Which protocols are accepted?
- Do they contain any ssl/tls vulnerabilities?

El auditor deberá ejecutar las herramientas sobre los siguientes dominios:

1. mypublicinbox.com
2. keysmartinc.com

Tras la ejecución, el objetivo es **comparar los resultados** obtenidos por cada herramienta (testssl.sh, SSLScan y SSL Labs) y responder a las siguientes preguntas clave de seguridad:

- ¿Qué protocolos SSL/TLS específicos son aceptados por el servidor en cada dominio?
- ¿Contienen los servidores alguna vulnerabilidad conocida dentro de SSL/TLS (como Heartbleed, POODLE, BEAST, etc.)?

Paso 1: Instalación y Propósito de las Herramientas de Auditoría

Para llevar a cabo la auditoría en la máquina virtual (VM), es necesario asegurarse de que las herramientas de terminal estén instaladas y comprender el propósito de cada una.

1. testssl.sh

- **Propósito:** Es un potente *script* en Bash que realiza un chequeo exhaustivo de protocolos, *cipher suites* y vulnerabilidades conocidas en la configuración SSL/TLS de un servidor. Su principal ventaja es el uso de **colores** en la consola para indicar visualmente el estado de seguridad (verde OK, naranja Advertencia, rojo Vulnerable).

Instalación: Si no está instalado por defecto en su distribución (como Kali Linux), puede instalarse mediante el gestor de paquetes.

```
sudo apt-get install testssl.sh
```

Sintaxis: El comando básico para la auditoría es simple, indicando la URL objetivo.

```
testssl.sh <dominio_objetivo>
```

Ejemplo: `testssl.sh mypublicinbox.com`

- (El comando `testssl` por sí solo sin argumentos muestra la ayuda con todas las opciones disponibles, como la opción `-U` para revisar todas las vulnerabilidades conocidas).

2. SSLScan

- **Propósito:** Es una herramienta de escaneo de línea de comandos rápida y concisa, diseñada principalmente para enumerar y listar las **suites de cifrado** (cipher suites) y los **protocolos** que acepta un servidor de destino. Es más rápida para el listado puro que *testssl.sh*.

Instalación: Esta herramienta también suele venir preinstalada en entornos de auditoría, pero si fuera necesario:

```
sudo apt-get install sslscan
```

Sintaxis:

```
sslscan <dominio_objetivo>
```

Ejemplo: `sslscan keysmartinc.com`

3. SSL Labs (Qualys)

- **Propósito:** Es una aplicación web, propiedad de Qualys, que realiza un **análisis profundo y externo** del servidor. El resultado es una **nota de calificación** (A, B, C, etc.) basada en la evaluación del certificado, el soporte de protocolos, el intercambio de claves y la robustez de los cifrados.
- **Procedimiento:** Se accede directamente al servicio *SSL Server Test* en el navegador. Se introduce la URL objetivo y se debe **asegurar** marcar la opción **"Do not show the results on the boards"** para que los resultados no queden indexados ni visibles en los paneles públicos de la herramienta.

Paso 2: Ejecución Práctica sobre los Dominios de Prueba

El ejercicio requiere la ejecución secuencial de estas tres herramientas sobre cada uno de los dominios: `mypublicinbox.com` y `keysmartinc.com`.

Ejecución para mypublicinbox.com

1. En la Terminal (testssl.sh):

```
testssl.sh mypublicinbox.com
```

- **Lo que hay que buscar:** Preste atención a las líneas en color **naranja** (advertencia) o **rojo** (vulnerable) para identificar protocolos obsoletos (SSLv2/v3, TLSv1.0/v1.1) o vulnerabilidades específicas (BEAST, Lucky13).

2. En la Terminal (SSLScan):

```
sslscan mypublicinbox.com
```

- **Lo que hay que buscar:** Revise el listado directo para ver si protocolos como **SSLv3** o **TLSv1.0** aparecen listados como "Accepted" (aceptados).

3. En el Navegador (SSL Labs):

- **Lo que hay que buscar:** Observe la **calificación general** (A, B, etc.) y la sección de *Configuration* para ver la lista de protocolos habilitados y la sección de *Vulnerabilities* para las comprobaciones de fallos conocidos.

Ejecución para *keysmartinc.com*

Se repite el mismo procedimiento para el segundo dominio, usando la sintaxis de comando correspondiente para cada herramienta.

Paso 3: Análisis y Comparativa de Resultados

El objetivo del ejercicio no es solo ejecutar comandos, sino **comparar y contrastar** cómo cada herramienta presenta la misma información.

1. Protocolos Aceptados:

- Compare la información de protocolos obsoletos: *testssl.sh* los marca en naranja, *SSLScan* los lista simplemente como "Accepted", y *SSL Labs* los penaliza en la nota final. Si un protocolo obsoleto (como **TLSv1.0**) es aceptado, se debe documentar.

2. Detección de Vulnerabilidades:

- Identifique si se reporta alguna vulnerabilidad SSL/TLS:
 - Si *testssl.sh* indica que el servidor es potencialmente **VULNERABLE** a **BEAST** o **Lucky13**, compruebe si esto se debe a que el servidor aún acepta el protocolo **TLSv1.0** (que usa el modo CBC vulnerable).
 - Verifique que vulnerabilidades críticas como **Heartbleed** (que ataca OpenSSL) o **POODLE** (que ataca SSLv3) sean marcadas como "not vulnerable (OK)".

3. Evaluación de Suites de Cifrado:

- Aunque no se pide explícitamente, utilice la información de las *cipher suites* proporcionada por las herramientas (especialmente por *SSL Labs* y *SSLScan*) para evaluar si el servidor permite el uso de cifrados débiles o *suites* que dependen de algoritmos que han demostrado fallos (como los que usan RC4 o los de 64 bits).

Este ejercicio práctico es crucial para pasar de la teoría a la aplicación, permitiendo al auditor comprender la importancia de utilizar múltiples fuentes para validar la seguridad criptográfica de un servidor.

Clickjacking

En esta sesión, nos adentraremos en el **Clickjacking**, una vulnerabilidad que se diferencia de otros ataques web tradicionales por estar dirigida directamente a engañar al usuario final, y no al sistema de la aplicación. También conocido como **UI Redress Attack** (Ataque de Revestimiento de Interfaz de Usuario), el Clickjacking es un ataque de ingeniería social basado en el engaño visual.

Concepto y Mecanismo del Clickjacking

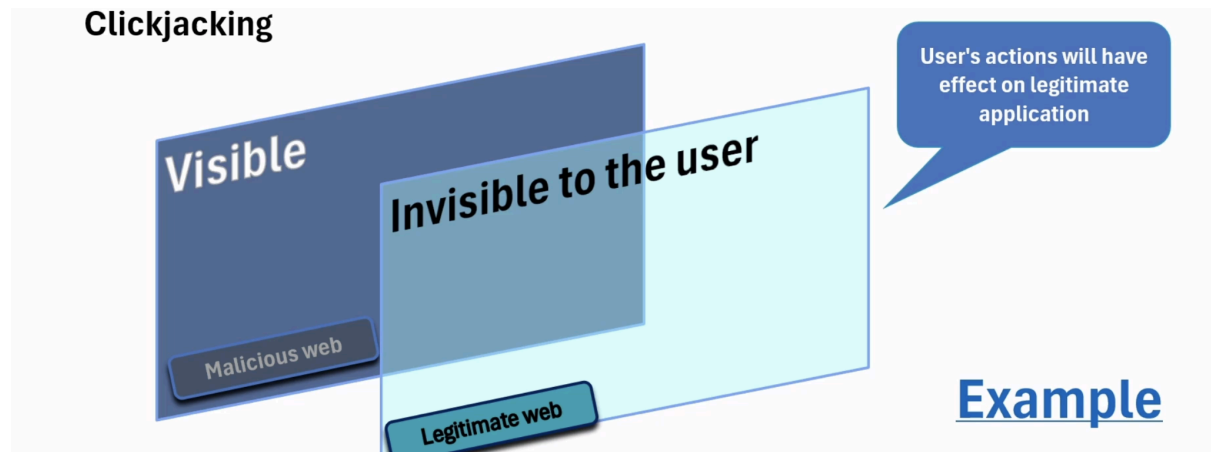
El objetivo principal del Clickjacking es **engañar al usuario para que haga clic en un elemento invisible de una aplicación legítima, creyendo que está interactuando con otra cosa**. Esto provoca que las acciones del usuario tengan un efecto diferente al esperado, a menudo con consecuencias perjudiciales.

A diferencia del **Cross-Site Request Forgery (CSRF)**, que explota una sesión activa del usuario para forzar una petición autenticada, el Clickjacking se aprovecha de la **percepción** del usuario y puede afectar tanto a clics como a la entrada de datos, independientemente de que la acción objetivo requiera autenticación o no.

El Uso de Capas HTML

Para lograr el engaño, el atacante utiliza la estructura de capas del lenguaje HTML, comúnmente mediante el uso de *iframes* (marcos internos).

- Se crea una **web maliciosa (capa visible)**, que es lo que el usuario ve y con lo que cree que está interactuando (por ejemplo, un juego o un vídeo).
- Se carga la **web legítima (capa invisible)** dentro de un `<iframe>`, y se coloca esta capa transparente, pero activa, justo encima del elemento que el atacante quiere que el usuario pulse (por ejemplo, un botón de "Aceptar" o "Eliminar").



Visualmente, el atacante presenta una **Web Maliciosa** visible, mientras que la **Web Legítima** se carga en una capa transparente e **invisible al usuario**. El clic del usuario, destinado a una acción en la capa visible (como matar un marcianito en un juego), pasa a través y activa una acción en la capa legítima (como borrar una cuenta). El resultado es que las **acciones del usuario tendrán un efecto en la aplicación legítima** sin que este lo sepa.

Ejemplo Visual de Explotación

El ataque de Clickjacking se entiende mejor con un ejemplo que demuestre cómo los clics se desvían de su propósito original.

- El usuario está participando en un juego donde debe hacer la mayor cantidad de clics en un botón para obtener una puntuación (por ejemplo, el vídeo con la URL <https://www.youtube.com/watch?v=gxyLbp1dmuU>).
- Cada clic que el usuario realiza para mejorar su puntuación en el juego (la capa visible) está, de hecho, registrando una acción en una capa transparente superpuesta (la web legítima).
- En el transcurso del juego, sin darse cuenta, el usuario puede estar activando acciones sensibles:
 - Hacer clic en un botón de confirmación invisible para **borrar todos los sitios web o datos** del usuario en la aplicación legítima.
 - Hacer clic en un botón invisible de confirmación para **dar permiso a la aplicación para acceder a su cámara web** ([Request Camera Access](#)).

Este ejemplo ilustra que el ataque no requiere que el usuario esté autenticado para ser efectivo, solo requiere que una acción sea posible mediante un clic, impactando la **confianza del usuario** en la aplicación y su seguridad.

Detección y Mitigación de Clickjacking

La forma de proteger una aplicación contra Clickjacking es **evitar que la página legítima pueda ser cargada dentro de un `iframe` externo**.

Para detectar si una página está protegida, debemos revisar las **cabeceras de seguridad** configuradas en la respuesta HTTP del servidor: `X-Frame-Options` o `Content-Security-Policy`.

1. Protección mediante X-Frame-Options (XFO)

Es una cabecera de seguridad antigua pero todavía utilizada para evitar que el contenido de la web se cargue en marcos de otras webs.

- **Sintaxis y Propósito:**

- **`X-Frame-Options: DENY`**: La política más restrictiva. La página no se cargará en un *iframe* en absoluto, independientemente del sitio que lo intente.
- **`X-Frame-Options: SAMEORIGIN`**: La página solo se cargará en un *iframe* si la página principal que lo contiene proviene del mismo dominio (mismo *origin*).

2. Protección mediante Content-Security-Policy (CSP)

CSP es una cabecera de seguridad más moderna y flexible. Para mitigar Clickjacking, se utiliza la directiva `frame-ancestors`.

- **Sintaxis y Propósito:**

- **`Content-Security-Policy: frame-ancestors 'none'`**: Equivalente a `X-Frame-Options: DENY`. Evita que la página se cargue en cualquier marco.

- **Content-Security-Policy: frame-ancestors 'self':**
Equivalente a **X-Frame-Options: SAMEORIGIN**. Solo permite la carga en marcos dentro del mismo origen.
- **Content-Security-Policy: frame-ancestors**
<dominio-específico>: Permite la carga en marcos solo desde una URL de confianza específica.

La corrección de esta vulnerabilidad es extremadamente sencilla (apenas 30 segundos a 2 minutos de trabajo) y consiste únicamente en configurar una de estas cabeceras correctamente.

Procedimiento Práctico de Detección Manual

Además de revisar las cabeceras con herramientas de navegación o un proxy (como Burp Suite), la forma más sencilla y visual de probar la vulnerabilidad es creando un archivo HTML de prueba localmente.

Creación del Archivo HTML de Prueba

El auditor debe crear un archivo con extensión **.html** que contenga una etiqueta **<iframe>** apuntando a la web objetivo.

Código de Prueba:

```
<html>
<head>
    <title>Clickjack page</title>
</head>
<body>
    <iframe src="http://www.target.site" width="700"
height="700"></iframe>
</body>
</html>
```

- **Sintaxis del Código:**

- La etiqueta `<iframe>` define una ventana incrustada dentro del documento HTML.
- El atributo `src` (`src="http://www.target.site"`) es la URL de la aplicación que se está auditando (la web legítima).
- Los atributos `width` y `height` (`width="700" height="700"`) definen las dimensiones del marco.

Lógica del Test

1. El auditor reemplaza `"http://www.target.site"` con la URL de la aplicación a auditar.
2. El auditor abre el archivo `.html` resultante en su navegador.
3. **Resultado de la prueba:**
 - Si la página web de la aplicación se **carga exitosamente** dentro del *iframe*, la aplicación es **vulnerable a Clickjacking**.
 - Si, por el contrario, aparece un cuadro en blanco o un mensaje de error del navegador, significa que la aplicación está **protegida** (gracias a `X-Frame-Options` o `Content-Security-Policy`).

Conclusiones sobre Clickjacking

Aunque esta vulnerabilidad no siempre tiene un impacto directo "súper grande" en términos de compromiso del servidor, afecta gravemente al **usuario** y a la **confianza** que este deposita en la aplicación. El usuario confía en que lo que ve es cierto y que la aplicación está protegida. Dada la **extrema facilidad tanto para detectarla como para corregirla**, es una vulnerabilidad que siempre debe ser reportada y priorizada para su solución inmediata por parte de los responsables de la aplicación.

Enunciado del Ejercicio Práctico sobre clickjacking

El ejercicio requiere la aplicación de este conocimiento para la detección manual:

1. **Crear un archivo .html** utilizando el código de prueba anterior para verificar la vulnerabilidad de Clickjacking.
2. **Revisar si los siguientes sitios web son vulnerables** a Clickjacking utilizando el archivo HTML creado:
 - 0xword.com
 - [Mypublicinbox.com](https://mypublicinbox.com)

Se recomienda a los estudiantes que, si ninguno de estos dominios resulta vulnerable, busquen una tercera página web (como la de un banco, un periódico favorito o una red social) y realicen la prueba para verificar su configuración de seguridad.

Resolución del Ejercicio de Clickjacking

El objetivo de este ejercicio es aplicar la técnica de detección manual de **Clickjacking** (o *UI Redress Attack*) creando un *archivo HTML local* y verificando si las aplicaciones objetivo cargan su contenido dentro de un `iframe`, lo cual confirmaría la vulnerabilidad.

Paso 1: Creación y Sintaxis del Código HTML de Prueba

Para realizar la prueba de Clickjacking, necesitamos un archivo sencillo que contenga la etiqueta `<iframe>`. Este archivo servirá como la **web maliciosa (capa visible)**, que intentará cargar la **web legítima** de forma incrustada y transparente.

A continuación, se muestra el código y la explicación de su sintaxis:

El código utilizado es el siguiente:

HTML

```
<html>

<head>

    <title>Clickjack page</title>

</head>

<body>

    <iframe src="http://www.target.site" width="700"
height="700"></iframe>

</body>

</html>
```

```
1 <html>
2   <head>
3     <title>Clickjack page</title>
4   </head>
5   <body>
6     <iframe src="http://www.target.site" width="700" height="700"></
  iframe>
7   </body>
8 </html>
```

Explicación de la Sintaxis y Propósito

- **Líneas 1-4 (<html>, <head>, <title>):** Estructura estándar de un documento HTML. El título "Clickjack page" es solo el nombre de la pestaña que verá el auditor.
- **Línea 6 (<iframe>):** Esta es la etiqueta central del ataque. <iframe> (Inline Frame) incrusta otro documento HTML dentro del documento actual.
 - **src="http://www.target.site":** El atributo **src** es fundamental. Aquí es donde se debe **pegar la URL de la aplicación web a auditar**. Si el servidor objetivo no bloquea esta carga, la vulnerabilidad existe.
 - **width="700" height="700":** Estos atributos definen las dimensiones del *iframe* incrustado en píxeles. Esto permite al atacante

redimensionar la capa transparente para alinearla perfectamente con un botón específico en la página legítima.

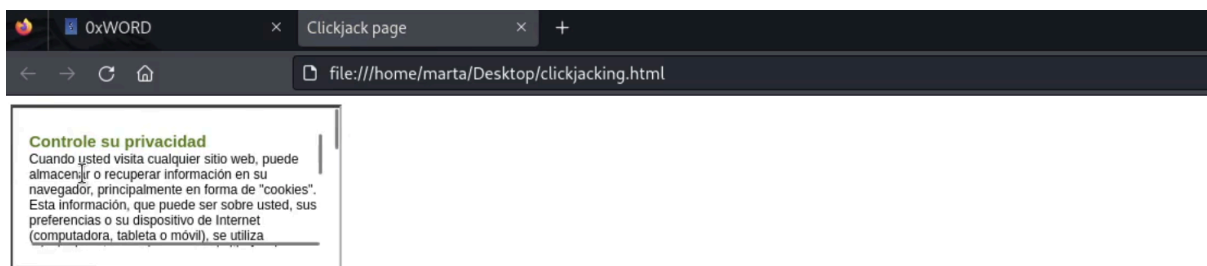
Procedimiento de Guardado:

El código debe guardarse en la máquina virtual (por ejemplo, en el escritorio) con un nombre descriptivo y la extensión `.html` (ej. `clickjacking.html`).

Paso 2: Ejecución de la Prueba sobre 0xword.com

El primer dominio a auditar es `0xword.com`. El procedimiento es sencillo y se realiza enteramente en el entorno local del auditor.

1. **Modificación del Código:** Se copia la URL `0xword.com` y se reemplaza el marcador de posición `"http://www.target.site"` en la línea 6 del código HTML por la URL del sitio. Se guarda el archivo modificado.
2. **Apertura Local:** El archivo `clickjacking.html` se abre desde el escritorio de la máquina virtual con un navegador web (la URL en la barra de direcciones del navegador mostrará el camino local:
`file:///home/marta/Desktop/clickjacking.html`).
3. **Análisis del Resultado:**



Al cargar el archivo, se observa que:

- La página de `0xword.com` se carga exitosamente dentro del marco (`<iframe>`).
- Incluso si el contenido no se muestra a gran escala o está incompleto, el hecho de que el servidor permita que el contenido aparezca demuestra que **el servidor no tiene configuradas las cabeceras de seguridad**

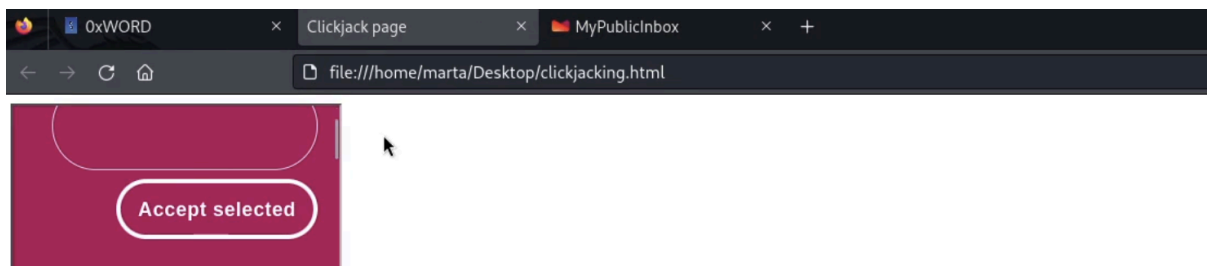
X-Frame-Options o Content-Security-Policy: frame-ancestors con las directivas restrictivas (DENY o 'none').

Conclusión para 0xword.com: La aplicación es **VULNERABLE** a Clickjacking. Un atacante podría superponer una capa transparente sobre elementos sensibles, como el botón "**Accept selected**" que se observa en la parte superior del *iframe* (probablemente relacionado con el consentimiento de cookies o privacidad).

Paso 3: Ejecución de la Prueba sobre mypublicinbox.com

El segundo dominio a auditar es mypublicinbox.com.

1. **Modificación del Código:** Se edita el archivo [clickjacking.html](#) nuevamente, reemplazando la URL de [0xword.com](#) por [mypublicinbox.com](#). Se guarda el cambio.
2. **Apertura Local y Análisis:** Se recarga la página [clickjacking.html](#) en el navegador.



Al cargar la página, se observa que:

- El contenido de **mypublicinbox.com** también se **carga exitosamente dentro del marco** (`<iframe>`).
- El tamaño de la ventana (`width="700" height="700"`) puede hacer que el contenido se vea pequeño o con barras de desplazamiento, pero la carga sin bloqueo es la prueba irrefutable de la falla.

Conclusión para mypublicinbox.com: La aplicación es **VULNERABLE** a Clickjacking, pues el servidor permite que su contenido sea incrustado en marcos de sitios de terceros.

Conclusión del Ejercicio y Utilidad de la Herramienta

Ambas páginas web probadas en el ejercicio (0xword.com y mypublicinbox.com) resultaron ser **vulnerables a Clickjacking** al no bloquear la carga de su contenido en un `iframe` externo.

La facilidad de este ejercicio resalta dos puntos cruciales en la auditoría web:

1. **Gravedad:** Aunque la explotación directa no siempre es fácil ni tiene repercusiones "graves" a nivel de servidor, el riesgo de explotación para realizar acciones en nombre del usuario (como consentimientos no deseados o gastos) existe.
2. **Practicidad:** Este archivo HTML creado es una herramienta de auditoría invaluable. Ahora, el auditor tiene un **archivo HTML reutilizable** que, simplemente cambiando la URL objetivo, permite verificar la vulnerabilidad de Clickjacking de cualquier aplicación web de forma **súper rápida** y sencilla, haciendo esta revisión una parte estándar y eficiente de cualquier auditoría de seguridad.

Cross-Site Request Forgery (CSRF): Falsificación de Petición en Sitios Cruzados

Bienvenidos a esta sesión centrada en el **Cross-Site Request Forgery (CSRF)**, una de las vulnerabilidades más comunes y críticas en las aplicaciones web. También conocida en español como **Falsificación de Petición en Sitios Cruzados**, este ataque se aprovecha de la confianza que una aplicación legítima deposita en el navegador de un usuario autenticado.

El concepto central del CSRF es que la víctima **envía una petición HTTP a una aplicación, sin ser consciente de ello, en la que previamente ha iniciado sesión**. La aplicación atacada ejecuta la petición en nombre de la víctima, asumiendo que es una acción legítima, lo que puede provocar un cambio de estado no deseado o la fuga de información.

1. Mecanismo y Diferencia con Clickjacking

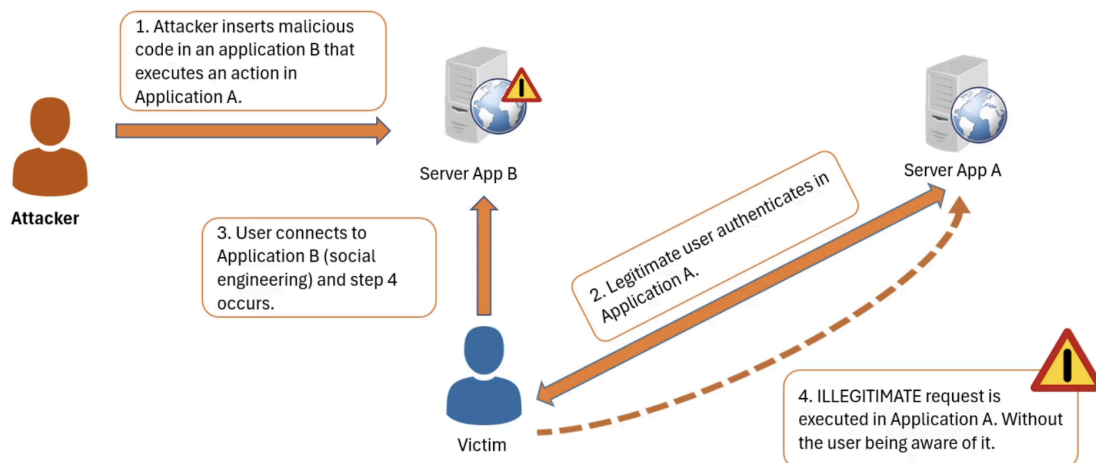
Es fundamental diferenciar CSRF de otras vulnerabilidades similares como el Clickjacking:

- **CSRF (Cross-Site Request Forgery):** Ataca la **confianza del sitio web** en el navegador y las *cookies* de sesión de la víctima. Requiere que la víctima esté **autenticada** en la aplicación atacada.
- **Clickjacking:** Ataca la **confianza del usuario** en la interfaz web (engaño visual). No necesariamente requiere autenticación, sino que explota la percepción del usuario.

Explicación Visual del Flujo de Ataque

El ataque CSRF sigue una secuencia de pasos que involucra al atacante, a la víctima y a dos aplicaciones (una legítima y una maliciosa).

CSRF



1. **El Atacante Inserta Código Malicioso (Aplicación B):** El atacante aloja el código malicioso (por ejemplo, una imagen invisible que dispara una petición HTTP o un formulario oculto) en un servidor externo o en una **Aplicación B**. Esta es la aplicación *maliciosa*.
2. **Usuario Legítimo se Autentica (Aplicación A):** La **Víctima** inicia sesión en la **Aplicación A** (la web legítima, como un banco o un webmail). El navegador guarda la *cookie* de sesión y el servidor confía en ese identificador.
3. **El Atacante Engaña a la Víctima:** Mediante ingeniería social (un enlace, un correo electrónico, una web engañosa), el atacante convence a la **Víctima** de visitar la **Aplicación B** (el sitio malicioso).
4. **Ejecución de Petición Ilegítima:** Al cargar la Aplicación B, el código malicioso se ejecuta y **fuera al navegador de la Víctima a enviar una petición** (la línea discontinua) a la Aplicación A. Dado que el navegador adjunta automáticamente la *cookie* de sesión de la Víctima, el **Servidor de la Aplicación A** interpreta la petición como legítima y la **ejecuta**, sin que la Víctima sea consciente de ello.

Ejemplo Práctico (Transferencia Bancaria): La víctima tiene sesión iniciada en su banco (Aplicación A). Un atacante le envía un *email* atractivo. Al hacer clic en el enlace (Aplicación B), el navegador envía automáticamente una petición al banco con la *cookie* de sesión de la víctima que dice: "Transfiere 1000 euros a la cuenta X". El banco lo ejecuta porque confía en la *cookie*.

2. Procedimiento Práctico: Explotación de CSRF en DVWA

Para comprender a fondo la vulnerabilidad, analizaremos un ejemplo real de cambio de contraseña en la aplicación de entrenamiento DVWA (Damn Vulnerable Web Application).

Configuración Inicial: La Petición Vulnerable

El primer paso de la auditoría es analizar la funcionalidad sensible (cambio de contraseña) utilizando un *proxy* de intercepción, como **Burp Suite**, para entender cómo se construye la petición legítima.

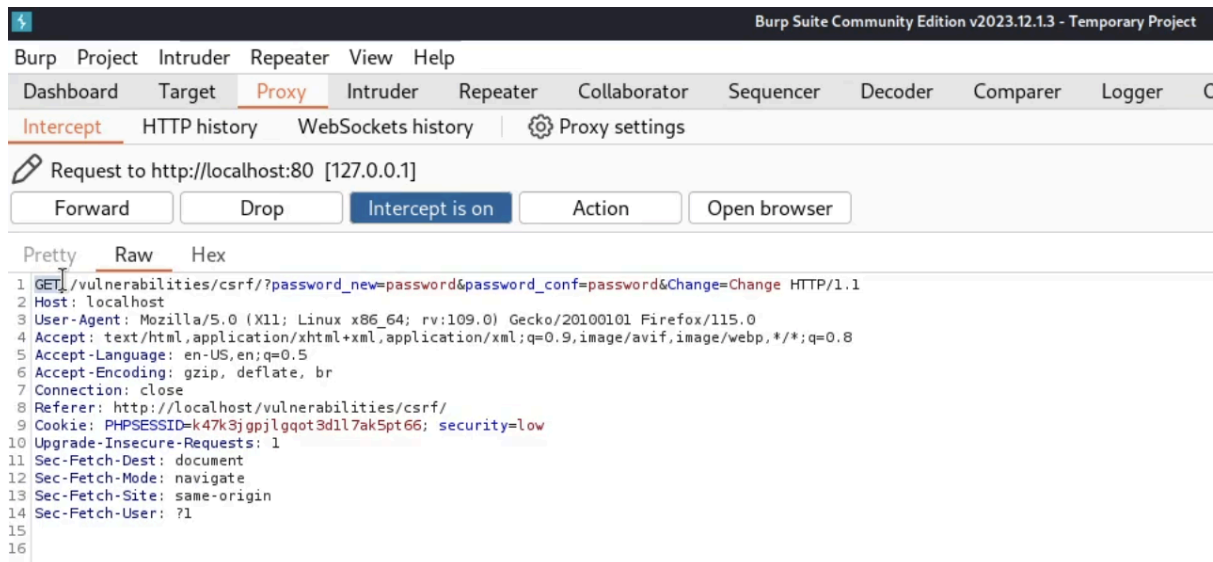
1. **Análisis del Formulario:** Al inspeccionar el código fuente de la funcionalidad de cambio de contraseña, encontramos el siguiente formulario:

HTML

```
<form action="#" method="GET">
  New password:<br />
  <input type="password" AUTOCOMPLETE="off"
name="password_new"><br />
  Confirm new password:<br />
  <input type="password" AUTOCOMPLETE="off"
name="password_conf"><br />
  <br />
  <input type="submit" value="Change" name="Change">
</form>
```

- **Punto de Falla Crítico (Método GET):** La línea clave es `method="GET"`. Este es un error fundamental de desarrollo, ya que las peticiones que modifican el estado del servidor (**cambios de contraseña, borrado, transferencia**) **NUNCA** deben usar el método `GET`.

- **Transmisión de Datos Sensibles:** Debido a que usa **GET**, los parámetros de la contraseña se envían en la propia URL (Query String), lo que, si no se usa HTTPS, expone la contraseña en texto claro.
2. **Petición Interceptada (Burp Suite):** Al intentar cambiar la contraseña a "admin" y capturar la petición con Burp Suite, se confirma la falla.



La petición capturada se ve así (no hemos incluido las cabeceras, sólo la petición):

GET

/vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Participar HTTP/1.1

Conclusión del Análisis: La vulnerabilidad es evidente. La petición se encapsula totalmente en la URL, y la única forma en que el servidor valida al usuario es a través de la **Cookie de Sesión** que el navegador adjunta automáticamente.

Paso 3: Construcción y Explotación del Código Malicioso

El atacante debe crear una página (por ejemplo, **csrf.html**) que replique la acción sin el conocimiento de la víctima.

1. **Creación del Formulario Malicioso (csrf.html):**

El atacante utiliza el código fuente del formulario y lo modifica para realizar dos cambios esenciales: **ocultar los campos de la contraseña e inyectar los valores de la nueva contraseña.**

Código HTML Final del Atacante (Con Enriquecimiento de Sintaxis):

```
<form action="http://192.168.111.142/vulnerabilities/csrf/?"
method="GET">
    <h1>Consigue el viaje de tus sueños valorado en
10.000€</h1>
    <input type="hidden" AUTOCOMPLETE="off"
name="password_new" value="0xword">
    <input type="hidden" AUTOCOMPLETE="off"
name="password_conf" value="0xword">
    <input type="submit" value="Participar" name="Change">
</form>
```

- **action="http://192.168.111.142/vulnerabilities/csrf/?"**: Se ha sustituido el *action* (# original) por la URL completa de la funcionalidad vulnerable en el servidor DVWA.
- **type="hidden"**: Este atributo hace que los campos de contraseña y confirmación no sean visibles para la víctima.
- **value="0xword"**: Se inyecta la contraseña deseada por el atacante (0xword).
- **Botón de Engaño**: El **value="Participar"** en el botón de *submit* engaña a la víctima.

2. Explotación (La Casuística Perfecta):

- El atacante sube **csrf.html** a un servidor externo.
- La víctima inicia sesión en DVWA (**admin/password**).

- La víctima accede a la página maliciosa `csrf.html` y, al ver la oferta del viaje, hace clic en el botón **"Participar"**.

3. Resultado Transparente:

- El clic en "Participar" lanza la petición **GET** al servidor DVWA.
 - El navegador adjunta la *cookie* de la víctima.
 - El servidor DVWA recibe: `GET /vulnerabilities/csrf/?password_new=0xword&password_conf=0xword...`
 - El servidor, al creer que el usuario autenticado (la víctima) solicitó la acción, ejecuta el cambio. La víctima ve un mensaje de "Password Changed" y se pregunta por qué, pero el atacante ya ha tomado control de la cuenta.
-

4. Impacto y Estrategias de Mitigación

El CSRF es un riesgo elevado porque, si se logra explotar, el atacante puede forzar la ejecución de acciones sensibles, como **cambios de credenciales, acceso a cuentas privadas, modificación o borrado de datos** (por ejemplo, conseguir que un webmail reenvíe correos mediante la adición de un filtro malicioso).

Mitigación Técnica (El Estándar)

La mejor manera de protegerse contra CSRF es rompiendo el modelo de confianza implícito, asegurando que solo las peticiones generadas por la propia aplicación sean válidas.

1. Tokens Anti-CSRF (Sincronizador Token Pattern):

- **Mecanismo:** El servidor genera un *token* **aleatorio, único y asociado a la sesión del usuario**. Este *token* se incluye como un campo oculto (`<input type="hidden">`) en todos los formularios sensibles.
- **Validación:** El servidor debe validar el *token* antes de ejecutar cualquier acción. Si el *token* no coincide o está ausente (lo que sucede

en un ataque CSRF), la petición es invalidada y denegada. Este *token* debería cambiar con cada acción para mayor seguridad.

- **Regla:** Implementar el *token* en todas las funcionalidades críticas (cambios de contraseña, *logout*, *checkout*, etc.).

2. Atributo SameSite en Cookies:

- **Mecanismo:** Configurar el atributo **SameSite** en las *cookies* de sesión (**PHPSESSID**, etc.).
- **SameSite=Strict:** Solo se permite enviar la *cookie* cuando se visita el dominio original. Si la petición procede de una aplicación externa (el sitio malicioso del atacante), la *cookie* no funciona, impidiendo que la petición se ejecute.

3. Verificación de Cabeceras (**Referer/Origin**):

- **Mecanismo:** El servidor revisa la cabecera HTTP **Origin** o **Referer** para asegurarse de que la petición procede del mismo dominio (origen) de la aplicación, denegando automáticamente las peticiones de aplicaciones externas.

Conclusiones sobre CSRF

El riesgo de CSRF es **muy elevado** en caso de éxito. Aunque la explotación requiere la casuística de que el usuario tenga la sesión iniciada en el momento del ataque (lo que dificulta la explotación masiva), la pérdida de reputación y confianza de cara a los clientes hace que la implementación del *token* anti-CSRF sea un requisito de seguridad **no negociable** en todas las funcionalidades que provoquen un cambio de estado.

Ejercicio Práctico Recomendado: Replicación del Ataque

Se propone como ejercicio de consolidación replicar el ataque de CSRF que acabamos de analizar en el entorno controlado de **DVWA (Damn Vulnerable Web Application)**.

Se invita a los estudiantes a:

1. Asegurarse de tener **Burp Suite** configurado como *proxy* de intercepción.
2. Crear su propia versión del archivo HTML (**csrf.html**), personalizando el código de ingeniería social.
3. Lanzar el ataque con la sesión de administrador de DVWA iniciada para observar en Burp Suite cómo se lanza la petición con la *cookie* de la víctima y verificar el cambio de contraseña.

El objetivo es entender de primera mano la **transparencia del ataque** para el usuario y la **simplicidad** con la que el servidor ejecuta la petición ilegítima.